NUnit Hands-On – Answers

1. Leap Year Rules (from the requirement)

The LeapYearCalculator.IsLeapYear(int year) method returns:

Return value	Condition
`1`	Year is a leap year
`0`	Year is not a leap year
`-1`	Year is out of valid range (< 1753 or > 9999)

Leap year logic (standard rules):

- Divisible by 4 → leap year candidate
- Divisible by 100 → NOT a leap year (exception to above)
- Divisible by 400 → IS a leap year (exception to exception)
- Valid range: 1753 to 9999 inclusive. Anything outside → return -1

2. Source Project – LeapYearCalculatorLib

The downloaded project contains LeapYearCalculator.cs. This is the class under test. Do not change this file unless asked to break the test.

LeapYearCalculatorLib/LeapYearCalculator.cs

```
namespace
LeapYearCalculatorLib { public
class LeapYearCalculator {
// Returns 1 if leap year
// 0 if not a leap year // -1 if year is out of
valid range (1753 - 9999) public int
IsLeapYear(int year) { // Validate range if
(year < 1753 || year > 9999) return -1;

// Apply leap year logic if
(year % 400 == 0) return 1;
if (year % 100 == 0) return
0; if (year % 4 == 0) return
1;

return 0;
}
}
}
```

The order of checks matters: 400 must come before 100, otherwise year 2000 would be incorrectly returned as 0 (not leap).

3. Test Project Setup – LeapYearCalculatorLib.Tests

- In the same solution: File → Add → New Project → Class Library (C#).
- Name it LeapYearCalculatorLib.Tests.
- Rename the default class file to LeapYearCalculatorTests.cs.
- Add a project reference to LeapYearCalculatorLib.

Install NuGet packages

Package Manager Console

```
Install-Package NUnit
Install-Package NUnit3TestAdapter
Install-Package Microsoft.NET.Test.Sdk
```

Test class skeleton

LeapYearCalculatorLib.Tests/LeapYearCalculatorTests.cs – skeleton

```
using NUnit.Framework;
using
LeapYearCalculatorLib;

namespace
LeapYearCalculatorLib.Tests {
[TestFixture] public class
LeapYearCalculatorTests { private
LeapYearCalculator _calc;

[SetUp] public void Init() {
_calc = new
LeapYearCalculator();
}

// --- test methods go here ---
}
}
```

4. Test Group 1 – Valid Leap Years (Expected: 1)

Use [TestCase] with year values that are known leap years. Each row is a separate test in Test Explorer. The method name follows the pattern: UnitUnderTest_Scenario_ExpectedOutcome.

Test cases chosen:

- 2000 – divisible by 400, so leap year
- 1904 – divisible by 4, not by 100, so leap year
- 2024 – divisible by 4, not by 100, so leap year
- 1600 – divisible by 400, so leap year

LeapYearCalculatorTests.cs – leap year tests (return 1)

```
// Parameterized test - valid leap years //
Method name: IsLeapYear_LeapYearInput_Returns1
[TestCase(2000)]
[TestCase(1904)]
[TestCase(2024)] [TestCase(1600)] public void
IsLeapYear_LeapYearInput_Returns1(int year) { int
result = _calc.IsLeapYear(year); Assert.That(result,
Is.EqualTo(1));
}
```

[Screenshot – Test Explorer: Leap Year Tests Passed]

```
Test Explorer - Leap Year (return 1)

Passed IsLeapYear_LeapYearInput_Returns1(2000) 6 ms
Passed IsLeapYear_LeapYearInput_Returns1(1904) 4 ms
Passed IsLeapYear_LeapYearInput_Returns1(2024) 3 ms Passed
IsLeapYear_LeapYearInput_Returns1(1600) 3 ms

4 / 4 passed
```

5. Test Group 2 – Non-Leap Years (Expected: 0)

Years that are not leap years should return 0.

Test cases chosen:

- 1900 – divisible by 100 but NOT by 400, so NOT a leap year
- 2001 – not divisible by 4, so NOT a leap year
- 1753 – boundary value, not divisible by 4
- 2023 – recent non-leap year

LeapYearCalculatorTests.cs – non-leap year tests (return 0)

```
// Parameterized test - non-leap years
[TestCase(1900)]
[TestCase(2001)]
[TestCase(1753)] [TestCase(2023)] public void
IsLeapYear_NonLeapYearInput_Returns0(int year) { int
result = _calc.IsLeapYear(year); Assert.That(result,
Is.EqualTo(0));
```

```
}
```

```
Test Explorer - Non-Leap Year (return 0)

Passed IsLeapYear_NonLeapYearInput_Returns0(1900) 5 ms
Passed IsLeapYear_NonLeapYearInput_Returns0(2001) 3 ms
Passed IsLeapYear_NonLeapYearInput_Returns0(1753) 3 ms
Passed IsLeapYear_NonLeapYearInput_Returns0(2023) 3 ms

4 / 4 passed
```

6. Test Group 3 – Invalid Year Range (Expected: -1)

Any year below 1753 or above 9999 should return -1.

Test cases chosen:

- 1752 – one below the lower boundary (1753)
- 0 – zero, clearly out of range
- -1 – negative year, out of range
- 10000 – one above the upper boundary (9999)

LeapYearCalculatorTests.cs – out of range tests (return -1)

```
// Parameterized test - invalid year range
[TestCase(1752)]
[TestCase(0)]
[TestCase(-1)] [TestCase(10000)] public void
IsLeapYear_OutOfRangeInput_ReturnsMinus1(int year) { int
result = _calc.IsLeapYear(year); Assert.That(result,
Is.EqualTo(-1));
}
```

[Screenshot – Test Explorer: Out of Range Tests Passed]

```
Test Explorer - Out of Range (return -1)

Passed IsLeapYear_OutOfRangeInput_ReturnsMinus1(1752) 5 ms
Passed IsLeapYear_OutOfRangeInput_ReturnsMinus1(0) 3 ms
Passed IsLeapYear_OutOfRangeInput_ReturnsMinus1(-1) 3 ms
Passed IsLeapYear_OutOfRangeInput_ReturnsMinus1(10000) 3 ms

4 / 4 passed
```

7. Test Group 4 – Boundary Value Tests

Boundary values are the most important values to test. We test exactly at 1753 and 9999 (both valid) and just outside (1752 and 10000, both invalid).

LeapYearCalculatorTests.cs – boundary tests

```
// Lower boundary: 1753 is valid, 1752 is not [TestCase(1753, 0)]
// 1753 is not a leap year but valid range -> 0
[TestCase(1752, -1)] // just below valid range -> -1

// Upper boundary: 9999 is valid, 10000 is not [TestCase(9999, 0)] // 9999
is not a leap year but valid range -> 0 [TestCase(10000,-1)] // just above
valid range -> -1 public void IsLeapYear_BoundaryValue_ReturnsExpected(int
year, int expected) { int result = _calc.IsLeapYear(year);
Assert.That(result, Is.EqualTo(expected));
}
```

[Screenshot – Test Explorer: Boundary Tests Passed]

```
Test Explorer - Boundary Values

Passed IsLeapYear_BoundaryValue_ReturnsExpected(1753, 0) 4 ms
```

```
Passed IsLeapYear_BoundaryValue_ReturnsExpected(1752, -1) 3 ms
Passed IsLeapYear_BoundaryValue_ReturnsExpected(9999, 0) 3 ms
Passed IsLeapYear_BoundaryValue_ReturnsExpected(10000,-1) 3 ms

4 / 4 passed
```

8. Breaking the Test – Modify Source and Rerun

The task asks us to deliberately break the source code and observe the test failures. We modify IsLeapYear() to always return 0.

LeapYearCalculator.cs – deliberately broken

```
public int IsLeapYear(int year) { //
Deliberately broken - always returns 0
return 0; }
```

[Screenshot – Test Explorer: Tests FAILED after breaking source]

```
Test Explorer - After Breaking the Source

Failed IsLeapYear_LeapYearInput_Returns1(2000)
Expected: 1 But was: 0

Failed IsLeapYear_LeapYearInput_Returns1(1904)
Expected: 1 But was: 0

Failed IsLeapYear_LeapYearInput_Returns1(2024)
Expected: 1 But was: 0

Failed IsLeapYear_LeapYearInput_Returns1(1600)
Expected: 1 But was: 0

Failed IsLeapYear_OutOfRangeInput_ReturnsMinus1(1752)
Expected: -1 But was: 0

Failed IsLeapYear_OutOfRangeInput_ReturnsMinus1(0)
Expected: -1 But was: 0

... (all tests that expected 1 or -1 fail)

Passed IsLeapYear_NonLeapYearInput_Returns0(1900) (still
passes) Passed IsLeapYear_NonLeapYearInput_Returns0(2001)
(still passes)

Passed: 4 Failed: 12 Total: 16
```

Revert the change and re-run

Restore the original IsLeapYear() logic in the source file and run again.

[Screenshot – Test Explorer: All Tests Green After Revert]

```
Test Explorer - After Reverting

Source All tests restored to green.

Passed: 16 Failed: 0 Total: 16 Test

Run Successful.
```

9. Complete Test File

All test methods in one file for reference.

LeapYearCalculatorLib.Tests/LeapYearCalculatorTests.cs – complete

```
using NUnit.Framework;
using
LeapYearCalculatorLib;
namespace
LeapYearCalculatorLib.Tests {
[TestFixture] public class
LeapYearCalculatorTests { private
LeapYearCalculator _calc;
[SetUp] public void Init() {
_calc = new
LeapYearCalculator();
}
// ■■ Group 1: Leap years -> return 1
[TestCase(2000)]
[TestCase(1904)]
[TestCase(2024)] [TestCase(1600)] public void
IsLeapYear_LeapYearInput_Returns1(int year) { int
result = _calc.IsLeapYear(year); Assert.That(result,
Is.EqualTo(1));
}
// ■■ Group 2: Non-leap years -> return 0
[TestCase(1900)]
[TestCase(2001)]
[TestCase(1753)] [TestCase(2023)] public void
IsLeapYear_NonLeapYearInput_Returns0(int year) { int
result = _calc.IsLeapYear(year); Assert.That(result,
Is.EqualTo(0));
}
// ■■ Group 3: Out of range -> return -1
[TestCase(1752)]
[TestCase(0)]
[TestCase(-1)] [TestCase(10000)] public void
IsLeapYear_OutOfRangeInput_ReturnsMinus1(int year) { int
result = _calc.IsLeapYear(year);
```

```
Assert.That(result, Is.EqualTo(-1));
}

// ■■ Group 4: Boundary values
[TestCase(1753, 0)]
[TestCase(1752, -1)]
[TestCase(9999, 0)] [TestCase(10000,-1)] public void
IsLeapYear_BoundaryValue_ReturnsExpected( int year,
int expected) { int result = _calc.IsLeapYear(year);
Assert.That(result, Is.EqualTo(expected));
}
}
}
```

10. Final Test Run – All 16 Tests

Open Test Explorer and click Run All. All 16 tests should be green.

[Screenshot – Test Explorer: All 16 Tests Passed]

```
Test Explorer - Full Run
========================

LeapYearCalculatorLib.Tests.LeapYearCalculatorTests

Passed IsLeapYear_LeapYearInput_Returns1(2000) 6 ms
Passed IsLeapYear_LeapYearInput_Returns1(1904) 4 ms
Passed IsLeapYear_LeapYearInput_Returns1(2024) 3 ms
Passed IsLeapYear_LeapYearInput_Returns1(1600) 3 ms

Passed IsLeapYear_NonLeapYearInput_Returns0(1900) 5 ms
Passed IsLeapYear_NonLeapYearInput_Returns0(2001) 3 ms
Passed IsLeapYear_NonLeapYearInput_Returns0(1753) 3 ms
Passed IsLeapYear_NonLeapYearInput_Returns0(2023) 3 ms

Passed IsLeapYear_OutOfRangeInput_ReturnsMinus1(1752) 4 ms
Passed IsLeapYear_OutOfRangeInput_ReturnsMinus1(0) 3 ms
Passed IsLeapYear_OutOfRangeInput_ReturnsMinus1(-1) 3 ms
Passed IsLeapYear_OutOfRangeInput_ReturnsMinus1(10000) 3 ms

Passed IsLeapYear_BoundaryValue_ReturnsExpected(1753, 0) 4 ms
Passed IsLeapYear_BoundaryValue_ReturnsExpected(1752, -1) 3 ms
Passed IsLeapYear_BoundaryValue_ReturnsExpected(9999, 0) 3 ms
Passed IsLeapYear_BoundaryValue_ReturnsExpected(10000,-1) 3 ms

========================================
Total: 16 Passed: 16 Failed: 0 Skipped: 0
Test Run Successful.
```